Perfect 👌 Since this is for **postgraduate NLP students**, I'll prepare **detailed, graduate-level lecture notes** with enough depth, technical explanations, and links to modern research directions.

Lecture Notes – Advanced Speech & Language Processing

---

1. Advanced Methods for Decoding

- **Definition**: Decoding = finding the most probable word sequence given an acoustic observation .

```
\hat{W} = \arg\max_W P(W|A) = \arg\max_W P(A|W)P(W)
```

- **Classical method**:
  - **Viterbi Algorithm** (dynamic programming): finds the best state sequence in Hidden Markov Models (HMM).
- **Limitations**: Large vocabulary → search space explosion.
- **Advanced Methods**:
  - **Beam Search Decoding**
    - Keeps only top-N hypotheses at each step (pruning).
    - Widely used in neural ASR and seq2seq models.
  - *A Decoding**
    - Uses heuristic cost function → faster convergence.
  - **Token Passing Algorithm**
    - Tokens represent hypotheses propagating frame by frame.
  - **Weighted Finite-State Transducers (WFSTs)**
    - Unifies acoustic, lexical, and language models in one graph.
    - State-of-the-art in HMM-DNN ASR systems.
  - **Neural End-to-End Decoding**
    - CTC (Connectionist Temporal Classification).
    - RNN-Transducers.
    - Attention-based seq2seq decoding (transformers).

---

2. Acoustic Processing of Speech

- Converts **raw waveform** → compact **feature vectors**.
- **Steps**:
  1. **Pre-emphasis** – High-pass filter to amplify high-frequency formants.
  2. **Framing & Windowing** – Speech is quasi-stationary in 20–30 ms windows.

3. **Fourier Transform** – Frequency representation.
4. **Filterbank Analysis** – Mel or Bark scale filters approximate human ear.
5. **Feature Extraction**:
    - **MFCC (Mel-Frequency Cepstral Coefficients)** – dominant in ASR.
    - **PLP (Perceptual Linear Prediction)**.
    - **Log-Mel Spectrograms** (for deep learning models).
    - **Modern Trend**: End-to-end ASR directly from raw waveforms (Wav2Vec 2.0, HuBERT).

---

3. Computing Acoustic Probabilities

- Goal: Compute **P(A|W)**, probability of signal given a word.
- **Traditional approach**:
    - **HMM + GMM** framework.
    - States model phonemes, GMMs model emission probabilities.
- **Modern approach**:
    - **Hybrid HMM-DNN**: DNN replaces GMM for state likelihoods.
    - **End-to-End Neural Models**:
        - **CTC-based**: align speech and text without explicit HMM.
        - **Attention models**: sequence-to-sequence mapping.
        - **Transformers**: self-attention captures long dependencies.

---

4. Training a Speech Recognizer

- **Pipeline**:
    1. **Data collection** – Thousands of hours of labeled audio.
    2. **Feature extraction** – MFCC/log-mel features.
    3. **Acoustic Model training** – HMM-DNN or end-to-end models.
    4. **Language Model training** – n-grams, RNN-LMs, Transformers (GPT-like models).
    5. **Pronunciation Lexicon** – Word-to-phoneme mappings (IPA or CMUdict).
    6. **Integration** – Decode using joint acoustic + language probabilities.
- **Optimization**:
    - Maximum Likelihood Estimation (MLE).
    - Discriminative training (MMI, MPE).
    - Cross-entropy or CTC loss.
    - Modern: **Self-supervised learning** (e.g., Wav2Vec2.0 pretraining).

---

5. Waveform Generation for Speech Synthesis

- **Text-to-Speech (TTS)** → Converts text into waveform.
- **Methods**:
  - **Formant Synthesis** – Rule-based (robotic).
  - **Concatenative Synthesis** – Uses prerecorded segments.
  - **Statistical Parametric (HMM-based)** – Predicts parameters → vocoder generates waveform.
  - **Neural Approaches**:
    - **WaveNet** – autoregressive raw waveform model (Google DeepMind).
    - **Tacotron 2** – seq2seq text-to-mel, vocoder for waveform.
    - **FastSpeech** – transformer-based, faster inference.
    - **VALL-E / VITS** – zero-shot voice cloning, end-to-end.

---

6. Human Speech Recognition

- **Biological Perspective**:
  - Cochlea → frequency analysis (like Fourier transform).
  - Auditory cortex processes phonetic/semantic content.
  - Humans adapt to noise, accents, incomplete input.
- **Comparison with ASR**:
  - Humans use **context + world knowledge** effectively.
  - Machines rely heavily on training data and statistical models.
  - Current research: **cognitive-inspired ASR** (multi-modal, memory-based).

---

7. Word Classes and POS Tagging

- **Word Classes (Parts of Speech)**: Noun, Verb, Adjective, Adverb, Pronoun, Preposition, Conjunction, Determiner, Interjection.
- **POS Tagging** = Assigning tags to words in a sentence.
  Example:
  - Sentence: *The dog chased the cat*
  - Tags: *DT NN VBD DT NN*

Tagsets for English:

- **Penn Treebank (45 tags)** – Standard in NLP research.
  - NN (noun singular), NNS (plural noun), VB (verb base), VBD (verb past tense), JJ (adjective), RB (adverb), DT (determiner), IN (preposition).

---

8. POS Tagging Methods

- **Rule-based**: Early systems (ENGTWOL, Brill tagger).

- **Statistical Models**:
  - Hidden Markov Models (HMM).
  - Maximum Entropy Models.
- **Neural Approaches**:
  - BiLSTM-CRF (sequence labeling).
  - Transformer-based (BERT, RoBERTa, GPT).
  - Fine-tuned models achieve >97% accuracy.
- **Applications**:
  - Parsing & Syntax Analysis.
  - Machine Translation.
  - Question Answering.
  - Information Extraction.

Summary

- **Decoding**: from HMM-Viterbi to Neural Beam Search.
- **Acoustic Processing**: MFCCs → self-supervised embeddings.
- **Acoustic Probabilities**: HMM-GMM → DNNs → End-to-End Transformers.
- **Training**: supervised + self-supervised trends.
- **Synthesis**: WaveNet, Tacotron, VALL-E.
- **Human vs. Machine Speech Recognition**: robustness gap.
- **POS Tagging**: from rule-based to transformers.

This set of notes is **detailed enough for PG-level NLP students** and links classical speech recognition to modern deep learning & transformer-based approaches.

Do you also want me to prepare a **research directions + recent papers section** (so students can explore latest work in speech recognition & synthesis)?

Great 👍 Let's expand on those **three classical approaches to POS Tagging** with enough depth for a **postgraduate NLP lecture**.

POS Tagging Methods

1. Rule-Based Part-of-Speech Tagging

- **Definition**: Uses **hand-crafted linguistic rules** to assign tags.
- **Process**:
  - Dictionary/lexicon provides possible tags for each word.
  - Contextual rules resolve ambiguity (e.g., noun vs. verb).
- **Example Rules**:

    - If the word is preceded by a determiner (DT) → likely a **noun**.
    - If the word ends with *-ing* → likely a **verb (VBG)**.
    - If the word ends with *-ly* → likely an **adverb (RB)**.
- **Example System**:
    - **ENGTWOL** (Constraint Grammar Tagger).
- **Advantages**:
    - Linguistically interpretable.
    - High accuracy for controlled domains.
- **Limitations**:
    - Requires extensive manual rule creation.
    - Not scalable to large, open-domain corpora.

---

2. Stochastic (Statistical) Part-of-Speech Tagging

- **Definition**: Assigns tags based on **probabilities learned from annotated corpora**.
- **Approach**:
    - Uses training data (e.g., Penn Treebank).
    - Learns probabilities of tags and sequences.
- **Models**:
    - **Unigram Tagger** – assigns most frequent tag for each word.
        - e.g., *book* → usually NN (noun).
    - **Bigram Tagger** – considers previous tag.
        - P(tag | previous tag).
    - **Trigram Tagger** – considers two previous tags.
    - **Hidden Markov Models (HMMs)**:
        - Word sequence = observations.
        - Tags = hidden states.
        - Uses **Viterbi Algorithm** for most likely tag sequence.
- **Example**:
    - Sentence: *The dog barks.*
    - Using probabilities, HMM assigns *DT NN VBZ*.
- **Advantages**:
    - Automatically learns from data.
    - Scalable to large corpora.
- **Limitations**:
    - Requires large tagged corpora.
    - Struggles with unseen words (OOV problem).

---

3. Transformation-Based Tagging (TBL)

- **Definition**: Hybrid between **rule-based** and **stochastic** approaches.
- **Invented by**: **Eric Brill (Brill Tagger, 1992)**.
- **Working**:
    1. Start with a baseline tagger (usually unigram frequencies).
    2. Apply **transformation rules** iteratively to fix mistakes.

3. Rules are **automatically learned** from training data.

- **Example Rule**:
  - *Change a word from VB → NN if the previous word is a determiner (DT).*
  - Applied to *"the book"*:
    - Initially → *DT VB*.
    - Rule corrects → *DT NN*.

- **Advantages**:
  - Rules are **interpretable** and **data-driven**.
  - More compact than purely statistical models.

- **Limitations**:
  - Slower training (needs rule learning).
  - Lower accuracy compared to modern neural models.

Comparison of POS Tagging Methods

Method	Basis	Example System	Pros	Cons
Rule-based	Hand-crafted rules	ENGTWOL	Transparent, linguistic knowledge	Labor-intensive, brittle
Stochastic (Statistical)	Probability models	HMM Taggers	Scalable, learns from corpora	Needs large data, OOV problem
Transformation-Based	Rule + learning	Brill Tagger	Interpretable, data-driven	Slower training, less accurate than neural

---

These three methods form the **foundation** of POS tagging before the **neural era** (BiLSTM-CRF, Transformers).

Would you like me to also **add Python code examples** (NLTK for rule-based, HMM tagger, Brill tagger) so your students can run experiments?

Perfect 👌 This is an advanced NLP topic — I'll prepare **detailed postgraduate-level lecture notes** on **Transformers and Large Language Models (LLMs)** with clear structure, theory, and connections to modern research.

Lecture Notes – Transformers & Large Language Models

---

1. The Transformer: A Self-Attention Network

- **Proposed by Vaswani et al. (2017): "Attention is All You Need"**.
- **Key Idea**: Replace recurrence (RNNs) and convolutions (CNNs) with **self-attention**.
- **Why?**
  - RNNs → sequential, slow, poor at long-range dependencies.
  - Self-attention → processes sequences **in parallel**, models **long-range dependencies directly**.
- **Self-Attention Mechanism**:
  - Each token attends to **all other tokens** in a sequence.
  - Input word embeddings transformed into:
    - **Query (Q)**, **Key (K)**, **Value (V)** matrices.
  - Attention score:

```
    \text{Attention}(Q,K,V) =
\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V
-  d_k  = dimension of key vectors (scaling factor).
```

---

2. Multi-Head Attention

- Instead of **one attention function**, use **multiple heads** (parallel attention).
- Each head learns different relationships:
  - Syntax (e.g., subject-verb agreement).
  - Semantics (e.g., coreference, long-range meaning).
- Formula:

```
  \text{MultiHead}(Q,K,V) =
\text{Concat}(\text{head}_1,...,\text{head}_h)W^O
```

- **Advantage**: Captures **different linguistic dependencies** simultaneously.

---

3. Transformer Blocks

- The Transformer is built from **stacked encoder/decoder blocks**.

Encoder Block

1. Multi-Head Self-Attention.
2. Add & Norm (residual connection + layer normalization).
3. Feed-Forward Network (2-layer MLP).
4. Add & Norm again.

Decoder Block

1. Masked Multi-Head Self-Attention (prevents seeing future tokens).
2. Encoder-Decoder Attention (attends over encoder outputs).
3. Feed-Forward Network.
4. Residual + Norm connections.

---

4. The Residual Stream View of the Transformer Block

- Introduced in **Anthropic's research** (2022) → explains how transformers process information.
- Instead of seeing layers as separate, think of a **residual stream**:
  - The hidden state flows forward unchanged, while **attention and MLPs add modifications**.
- Residual connections = "shortcut pathways" that **enable deep transformers to train** by avoiding vanishing gradients.
- Provides **interpretability**:
  - Attention heads act like *routers* — deciding what info to pass.
  - MLPs act like *feature transformers*.

---

5. The Language Modelling Head

- A **language model (LM)** predicts the next token given previous context.
- Transformer-based LMs use:
  - **Masked Self-Attention** in decoder (causal).
  - **Softmax over vocabulary**:

```
P(w_t|w_1,...,w_{t-1}) = \text{softmax}(h_t W^T)
```

- Training objective: **minimize cross-entropy loss**:

```
L = -\sum_t \log P(w_t | w_{<t})
```

---

6. Large Language Models with Transformers

- **Scaling Laws** (Kaplan et al., 2020):
  - Performance improves predictably with **more parameters**, **more data**, and **more compute**.
- **Examples**:
  - GPT family (decoder-only).
  - BERT, RoBERTa (encoder-only).
  - T5, BART (encoder-decoder).
- **Trends**:
  - Billions to **trillions of parameters**.
  - Trained on massive web-scale corpora.
  - Emergent abilities: reasoning, coding, summarization, multi-modal tasks.

---

7. Large Language Models: Training Transformers

- **Training Objective**: Autoregressive LM (GPT), Masked LM (BERT), Seq2Seq LM (T5).
- **Data**:
  - Billions of tokens (Common Crawl, books, Wikipedia, code, dialogues).
  - Pretraining → Finetuning (domain-specific).
- **Optimization**:
  - Adam / AdamW optimizer.
  - Gradient clipping, mixed precision (FP16/FP32).
  - Distributed training (model parallelism, data parallelism, pipeline parallelism).
- **Challenges**:
  - **Compute cost** (hundreds of GPUs for months).
  - **Bias & fairness** in training data.
  - **Interpretability** (black-box problem).
  - **Efficiency**: pruning, quantization, distillation.

Summary

- **Transformers** replaced RNNs via **self-attention**.
- **Multi-head attention** lets the model learn diverse dependencies.
- **Transformer blocks** combine attention + feed-forward + residuals.
- **Residual stream view** explains interpretability.
- **Language modelling head** turns transformers into powerful generative models.
- **LLMs** (GPT, BERT, T5) are scaled-up transformers trained on massive corpora.
- **Training LLMs** requires data, compute, optimization tricks, and raises challenges (bias, cost, interpretability).

This set is detailed enough for **postgraduate NLP students** and can serve as both **theory + discussion base** for research-oriented classes.

Would you like me to also **add diagrams** (like a transformer block architecture & self-attention mechanism) to use directly in your slides?